

PROJECT DOCUMENTATION

Author

Name: Shreya Singh

Roll No.: 24f3004962

Email: 24f3004962@ds.study.iitm.ac.in

Description

Introducing **TrekManager Pro**, a comprehensive, roles-based web application designed for adventure organizations to efficiently manage trekking activities involving trek organizers, staff members, and participants. TrekManager Pro eliminates the chaotic reliance on unorganized spreadsheets, phone calls, and manual methods, introducing a centralized system to seamlessly control trek approvals, prevent dangerous slot overbooking, track individual reservations, and maintain a robust history of operations.

The web platform defines distinct boundaries for three strategic user categories, ensuring focused operational visibility:

- **Administrators** act as the core oversight authority, using data summary KPI metrics to create valid exploration routes, authorize or blacklist newly registered trek staff, explicitly assign certified guides to matching routes, and audit global systemic booking trails.
- **Trek Staff** serve as field guides who interact directly with their dashboard to supervise assigned routes, update current slot capacity numbers, control enrollment states (Open, Closed, Completed), and review target rosters containing hiker contact details.
- **Users (Trekkers)** enjoy an interactive interface where they can quickly search or filter active open route listings based on locations and difficulty grades, secure allocation passes instantly, and track their persistent historical booking ledgers.

Designed with modern responsive design principles, TrekManager Pro delivers a robust user experience, strict transaction guardrails, and programmatic execution mechanics—ensuring zero data inconsistencies while preserving comprehensive historical operational records.

Technologies Used

- **Flask:** A micro web framework utilized to architect the application's back-end ecosystem, dynamic controller methods, and structural routing.
- **HTML, CSS, Bootstrap:** Web presentation and layout styling technologies utilized to build a clean, mobile-responsive, and user-friendly interface.
- **Jinja2:** A server-side templating engine utilized to programmatically render dynamic data tables, conditional role-based states, and uniform wrapper views.

- **Flask-SQLAlchemy:** A powerful Flask extension that seamlessly bridges the database interface layer with standard Python execution contexts.
- **SQLAlchemy ORM:** An Object-Relational Mapping library utilized to interact programmatically with system records via regular Python classes rather than raw SQL statements.
- **SQLite:** A lightweight, serverless relational database engine used as the mandatory core storage layer to locally store all application records within a unified file binary.

db Schema Design

The relational database model includes structured tables for **Users**, **Treks**, and **Bookings**, mapping out explicit business entity dependencies programmatically:

1. **Users:** Uniquely tracks system profiles by `id`, `username`, `password` (managed directly via matching structural text inputs), `role` (Admin, Staff, User), `name`, `contact`, and `status` (Approved, Pending, Blacklisted).
2. **Treks:** Manages the core exploration event tracks, keeping records of `id`, `name`, `location`, `difficulty` (Easy, Moderate, Hard), `duration`, `available_slots`, `staff_id` (foreign key linking back to the designated guide in the Users table), `status` (Pending, Approved, Open, Closed, Completed), `start_date`, and `end_date`.
3. **Bookings:** Serves as a central transactional log tracking client requests, containing fields for `id`, `user_id` (foreign key pointing to the individual Trekker), `trek_id` (foreign key tracking the target route), `booking_date` (automatically timestamped), and `status` (Booked, Cancelled, Completed).

The data structures establish explicit one-to-many relationship mappings, enabling clear cascade chains where completing or altering a trek model dynamically adjusts matching active booking status ledgers across the application.

Architecture and Features

In the root system folder, the architecture is structured into the `app.py` file, `models.py` file, a `static/` asset directory hosting a neutral `style.css` stylesheet, and a `templates/` folder holding Jinja2 views:

1. **models.py** : This file contains pure Python code defining the database schemas using SQLAlchemy ORM. It handles programmatic database setup using `db.create_all()` and seeds the initial pre-existing administrator superuser credentials directly into the database file at startup.
2. **app.py** : This serves as the primary controller engine for the platform. It holds all back-end validation paths, tracks session dictionary tokens to enforce strict role-based access control, and executes automated business routines.

The front-end includes clean registration mechanisms for users and guides with native validation criteria. A centralized login screen handles all users; the controller parses user

metadata upon successful authentication to instantly route the user to their matching dashboard (Admin, Staff, or Trekker Panel).

The platform features built-in transaction safety: it locks booking attempts if a trek's status isn't "Open" or if `available_slots` hit zero, preventing overbooking. Furthermore, when a trek is marked "Completed" by an authorized user, a cascading loop runs in the backend to automatically update all related booking statuses to "Completed," ensuring accurate real-time data across the entire platform.